

Unit-I Fundamental of Microcontroller

I) Introduction

The microprocessor is one of the most important components of a digital computer. It acts as a brain of a computer system. A digital computer is a programmable machine. Its main components are: CPU (Central Processing Unit), memory, input device and output device as shown in Figure 1 below.

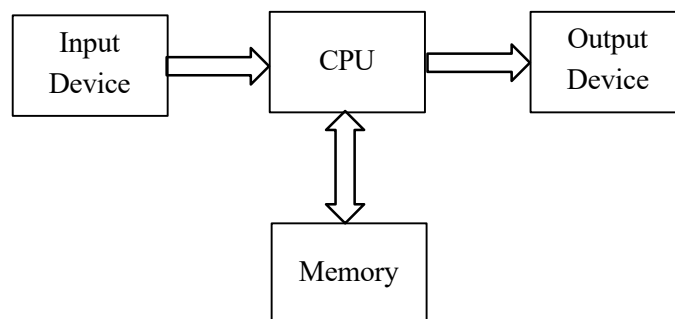


Figure 1: Schematic Diagram of Digital Computer.

The CPU executes instructions given by the programmer.

The input device is used to feed programs and data to the computer.

The memory is storage device. It stores programs, data and result.

The output device displays or prints programs, data and/or results according to the instruction given to the computer.

“The Central Processing Unit (CPU) built on single IC is called microprocessor.”
or “A microprocessor is a multipurpose, programmable, clock driven, register based electronic device that reads binary instructions from storage device called memory, accepts binary data as input and processes data according to those instructions, and provides results as output.”

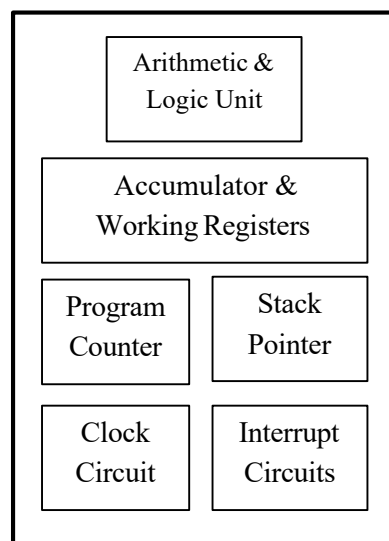


Figure 2: Block Diagram of Microprocessor.

Microcontroller (MCU) is like a small computer on a single Integrated Circuit (IC) that executes a user program for the purpose of controlling some device, hence the name microcontroller.

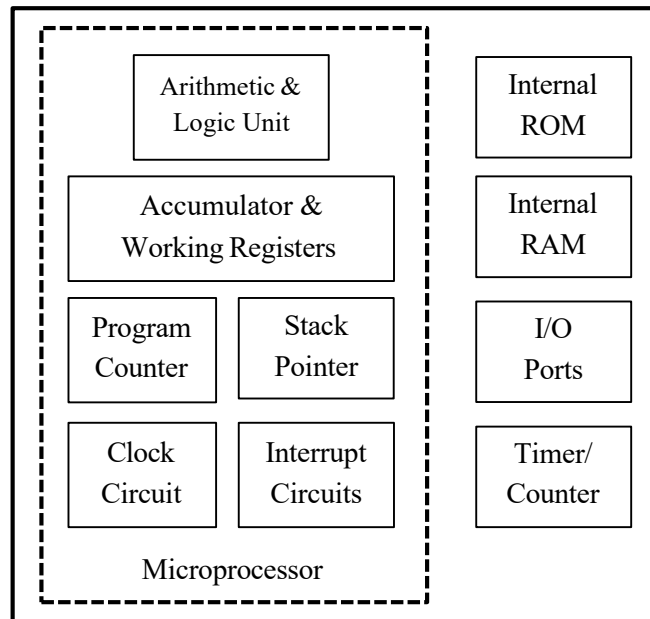


Figure 3: Block Diagram of Microcontroller

- All the components of the microprocessor such as ALU, SP, PC, ACC & working registers, clock circuit and interrupts circuit.
- In addition microcontroller has needed to Complete microcomputer system such as memories (ROM & RAM), Timers/Counters and I/O Ports.
- As microcontroller has all the components needed in its single chip, it does not need any external circuits to do its task, so microcontrollers are heavily used in embedded systems. A microcontroller can be called the heart of embedded system.

Difference between Microprocessor and Microcontroller:

Microprocessor	Microcontroller
It is heart of microcomputer system.	It is a heart of embedded system.
It is simply a processor without memory and I/O components and have to be connected externally.	It has processor along with internal memory and I/O components.
Since memory and I/O has to be connected externally, the circuit becomes large.	Since memory and I/O are present internally, the circuit is smaller.
Cannot be used in compact systems and hence inefficient.	Can be used in compact systems and hence it is an efficient technique.
Cost of the entire system increases.	Cost of the entire system is low
Due to external components, the entire power consumption is high.	Since internal components, total power consumption is less.

Since memory and I/O components are all external, each instruction will need external operation, hence it is relatively slower	Since components are internal, most of the operations are internal instruction, hence speed is fast.
It has less number of internal registers, hence more programs and operations are memory based.	It has more number of registers on-chip, hence the programs are easier to write.
Microprocessors are based on Von Neumann architecture <u>where program and data are stored in same memory module.</u>	Microcontrollers are based on Harvard architecture <u>where program memory and Data memory are separate.</u>

1.1 Features of 8051 microcontroller

- The microcontroller is **8 bit**.
- Designed using HMOS technology.
- In system programming (ISP) facility. Two lines TX and Rx from port 3 are used for serial programming.
- *4 KB internal program memory (ROM).*
- **128 bytes internal memory (RAM).**
- 40 pin DIP model and operated on +5 Volt power supply.
- Operating frequency 11.0592M Hz (12 MHz).
- 8 bit data bus and **16 bit address bus**.
- 32 general purpose registers: each of 8 bits.
- Two timers, Timer 0 and Timer 1, configured in 4 modes. 16 bit timers/counters.
- 3 internal and 2 external (INT0 and INT1) interrupts.
- 8 bit CPU with registers A and B.
- 16 bit Program Counter (PC) and Data Pointer (DPTR).
- 8 bit Program Status Word (PSW) and Stack Pointer (SP).
- Four register banks, each of 8 registers.
- Available in ROM, EPROM, EEPROM, version.
- Can access 64K byte program and 64K byte data memory externally.
- Multiply and divide instruction are available..
- 32 I/O lines spread over 4 ports – P0, P1, P2 and P3 of 8 bits wide.
- 16-bit address bus. However, lower orders address bus is multiplexed with data bus..
- It is having CISC instruction sets.

1.2 Block Diagram of 8051

The following figure 4 shows block diagram of Microcontroller 8051

Central Processing Unit (CPU):

- The CPU is the brain of processing device called microcontroller.
- It monitors and controls all operations that are performed in microcontroller..

Interrupts:

- Interrupt is a subroutine call that interrupts the microcontroller's main operations or work and causes it to execute any other program, which is more important at the time of main operation.
- . An Interrupt provides a mechanism to put on hold the on-going operations, execute a subroutine and then again resumes to another type of operations.
- Generally **five interrupt sources** are there in 8051 Microcontroller, these are
- 2 external interrupts,
- 2 timer interrupts,
- 1 serial port interrupts.

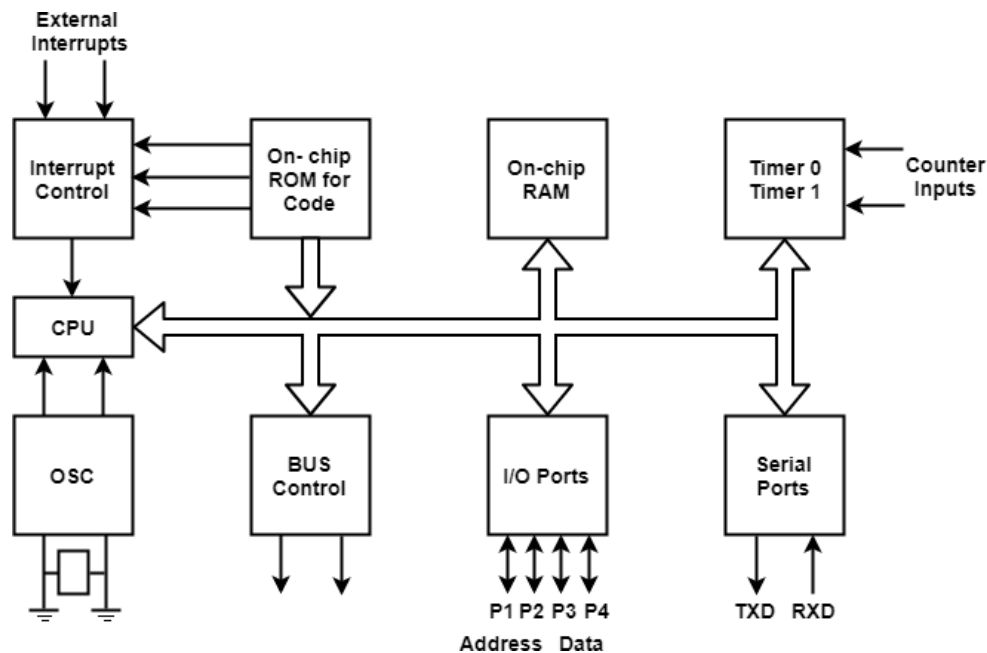


Figure 4: Block Diagram of 8051 Microcontroller.

- **Memory:**

- Microcontroller requires a program which is a collection of instructions.
- This program tells microcontroller to do specific tasks of specific operations.
- These programs require a memory on which these can be saved and read by Microcontroller to perform a particular task.

The memory which is used to store the program of microcontroller is known as **code memory or Program memory**.

-It is known as Read Only Memory (ROM).

Again to store the data temporarily for the various operations the memory which is used known as **Random Access Memory (RAM)**.

Oscillators: As we know microcontroller is a digital circuit hence it requires clock for frequency is **11.0592 MHz**

Input/Output (I/O) Ports:

8051 microcontroller has 4 I/O ports to connect other peripherals viz. Port 0 (P0), Port 1 (P1), Port 2 (P2) and Port 3 (P3). Each port is bit addressable as well as Byte addressable

Serial Port:

- It is full duplex type, i.e. it can transmit and receive simultaneously. Pin TXD is used to transmit the data and RXD is used to receive the data.
- The register SBUF is used to hold the data.

Timers/Counters:

Microcontroller has **two 16 bit** timers and counters.

- They are Timer0 (T0) and
- Timer1 (T1).
- These two counters/timers can be used either as timer to generate a time delay or as counter to count events happening outside the microcontroller.
- Since 8051 has 8 bit architecture, each Timer is accessed by two separate 8 bit special function registers viz. TL0, TH0, TL1 and TH1

ALU: This unit is for arithmetic and logical operations on 8 bit operands.

Accumulator (A or ACC): It is 8 bit addressable used by all arithmetic and logical instructions for their operations.

B (register): It is 8 bit register. The B register is used only for two multiplication and division operations but it is available as general purpose register when it is not used by multiplication and division operations.

- *Program counter (PC):*
 - It is a 16 bit register.
 - By this 8051 can access program address from 0000H to FFFFH a total of 64 KB of code.
 - It is used to hold the address of a byte in memory.

PSW registers:

- Flag bit register in 8051 is called as Program Status Word (PSW).
- It is an 8 bit and bit addressable special purpose register, means each bit can be set or reset independently.
- There are 4 flag in 8051, which are as below.
 - **P- Parity flag-** It indicates number of 1's in ACC. If it is 1 then odd number of 1 in accumulator and if it is 0 then even number of 1 in accumulator.
 - **O- Overflow flag:** Used to detect error in signed arithmetic operations. This flag is also be set along with carry flag.
 - **C- Carry flag:** It is used to detect carry out of D7 bit of result, after certain arithmetic operations.
 - **A- Auxiliary Carry:** It is used to check carry in D3 bit due to arithmetic operations. So it is useful in BCD arithmetic.

- *Stack and Stack Pointer (SP):*

-Stack is refers to an area of internal RAM which is used in conjunction with certain opcodes to store and retrieve data quickly.

- The SP is set to 07H after the reset.

Bus: Basically Bus is a collection of wires which work as a communication channel or medium for transfer of Data.

Address Bus: Microcontroller 8051 has a 16 bit address bus for transferring the data.

Data Bus: Microcontroller 8051 has 8 bits of the data bus, which is used to carry data of particular applications between the devices.

1.3 Pin description of 8051

- The INTEL 8051 is an 8-bit microcontroller with 128 Byte internal RAM and 4KB internal ROM.
- The 8051 is a pin 40 pin IC available in Dual-In Line (DIP) package.
- It requires a single power supply of +5V, where a total of 32 pins are set aside for the 4 ports P0, P1, P2 and P3.
- Each port takes 8 pins. The remaining 8 pins

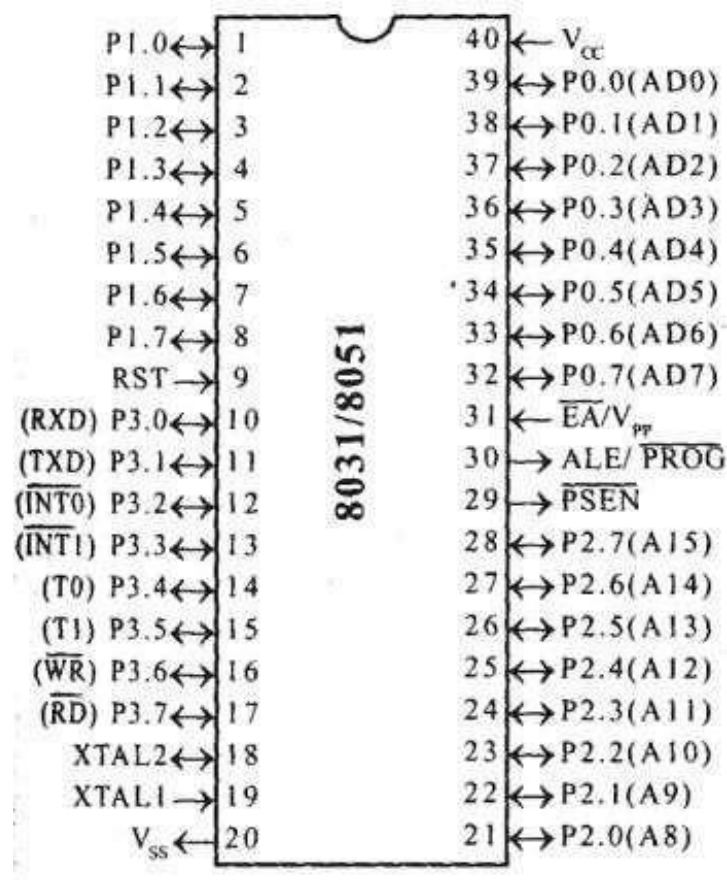


Figure 1.5: Block Diagram of 8051 Microcontroller.

- **Vcc:** Pin 40 & 20 provides the voltage supply +5 v usually.

- **Ports:** Microcontroller 8051 has 32 I/O lines spread over **four ports** .

i) Port 0 :

- Port 0 has total of 8 pins (Pin 32 to Pin 39). It can be used for input or output.
- each pin must be connected externally to a 10 K ohm pull-up register
- This is due to the fact that P0 is an open drain
- * when port make output pin status= 0
- * when port make input pin status= 1
- (* All ports P0,P1,P2 and P3 as its same)

ii) Port 1:

- - Port 1 has total of 8 pins (Pin 1 to Pin 8). It can be used as input or output.
- - This port doesn't need any pull up registers, since it already has pull up register, internally

Port 2 :

- Port 2 has total of 8 pins (Pin 21 to Pin 28). It can be used as input or output

Port 3.

- - It has total of 8 pins (Pin 10 to Pin 17). It can be used as input or output

- **PSEN (active low):** Program Strobe Enable (PSEN) pin 29 is output control pin.
 - **ALE:** - Address Latch Enable (ALE) Pin 30 is used to de-multiplex the low byte address or data using an external latch by connecting to the ground pin of the system.
 - - This is an output pin which is active High.
 - **EA (active low)/Vpp:** External Access (EA) Pin 31, when held Low then the microcontroller access program from external memory.
 - **XTAL1 AND XTAL2:** The 8051 has an on-chip oscillator but requires an external clock to run it. So, XTAL1 (Pin 19) and XTAL2 (Pin 18) pins are provided for external Clock frequency crystal of 11.0592 MHz.
 - **RST (active low):** The RST (low), Pin 9, signal is used to reset the microcontroller
- 1.3: The Total 40 pin functions of 8051.

1.4 Memory organization of 8051

The 8051 has a separate memory space for code (programs) and data handling. The show on-chip memory and external memory organization for 8051.

- The code memory and data memory are separate units the architecture is referred to as **Harvard architecture** whereas code memory and data memory are on a single unit that is sharing common of memory the architecture is referred as **Von Neumann** or Princeton architecture.

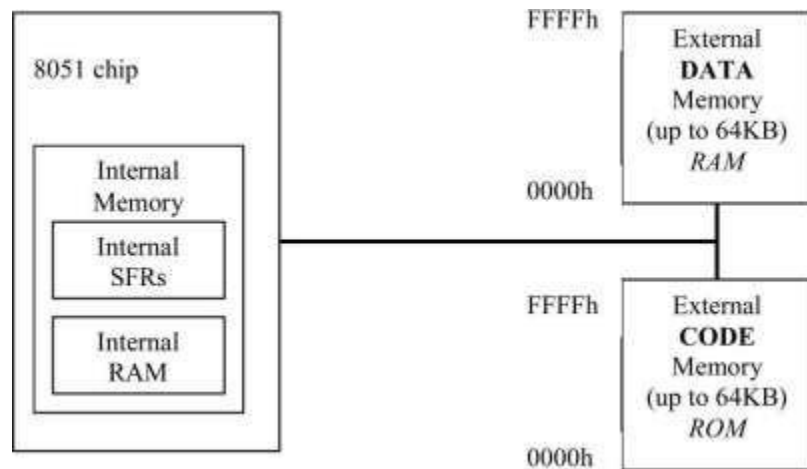


Figure 1.7: Memory representation of 8051 Microcontroller.

Program Memory (ROM)

In 8051, the code or instructions to be executed are stored in the Program Memory called as the ROM of the microcontroller. The original 8051 Microcontroller by Intel has 4KB of internal ROM.

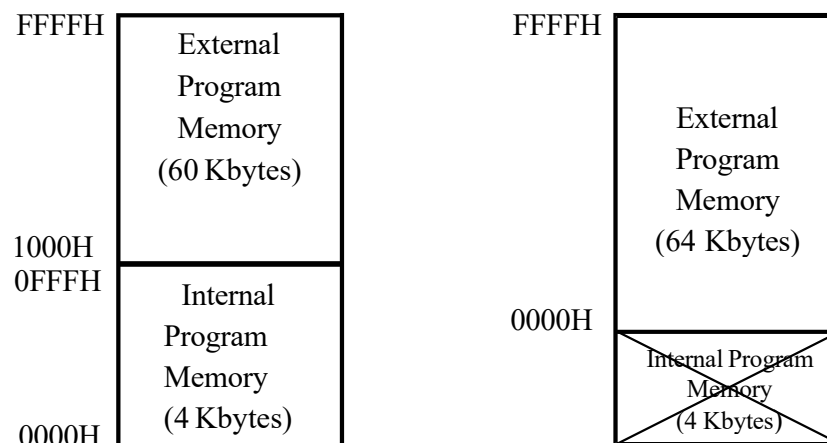


Figure . ROM Memory representation of 8051.

Data Memory or Internal RAM

The Data Memory or RAM of the 8051 Microcontroller stores temporary data and intermediate results that are generated and used during the normal operation of the microcontroller. The 8051 had 128 Byte of internal RAM. Register Banks or Working Registers (00H to 1FH)

1. Bit Addressable Area (20H to 2FH)
2. General Purpose RAM or Scratchpad Area (30H to 7FH)

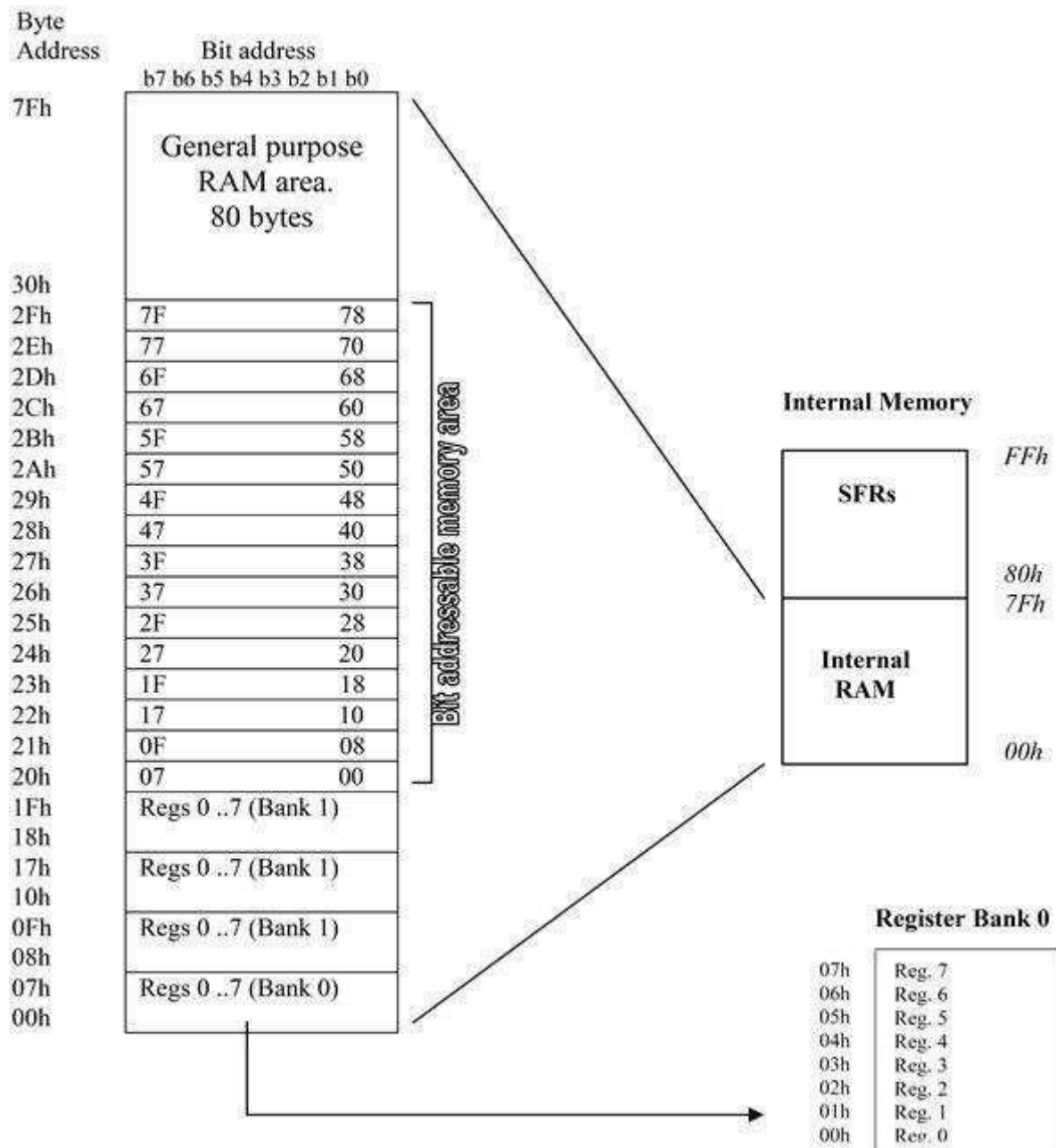
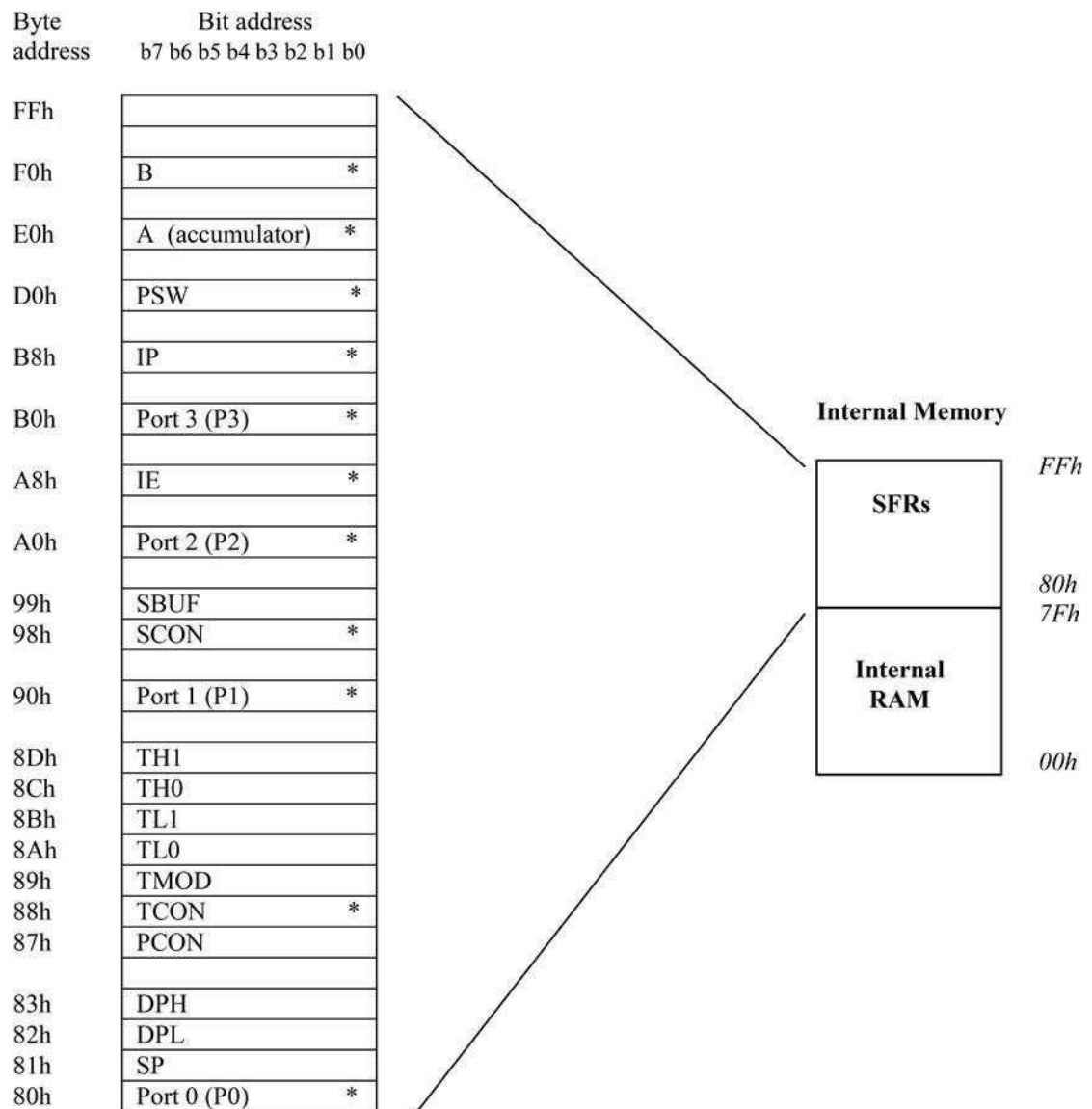


Figure 1.9: ROM Memory representation of 8051.

Special Function Registers (SFRs):

The upper 128B of the RAM, memory addresses from 80H to FFH, is allocated for a group of specific internal registers called Special Function Registers (SFRs)



* indicates the SFR registers which are bit addressable

Figure SFRs Memory representation of 8051.

Table 1.4: The SFRs of the 8051.

Name	Function	Internal RAM Address in Hex
A*	Accumulator	0E0H
B*	B-Register	0F0H
P0*	Port 0 Latch	80H
P1*	Port 1 Latch	90H
P2*	Port 2 Latch	0A0H
P3*	Port 3 Latch	0B0H
IP*	Interrupt Enable Control	0B8H
IE*	Interrupt Enable control	0A8H
TMOD	Timer/Counter Mode Control	89H
TCON*	Timer/Counter Control	88H
TH0	Timer/Counter 0(High Byte)	8CH
TL0	Timer/Counter 0(Low Byte)	8AH
TH1	Timer/Counter 1(High Byte)	8DH
TL1	Timer/Counter 1(Low Byte)	8BH
SCON*	Serial Port Control	98H
SBUF	Serial Port Data Buffer	99H
PCON	Power Control	97H
PSW*	Program Status Word	0D0H
SP	Stack Pointer	81H
DPH	Data Pointer (High Byte) for Addressing External Memory	83H
DPL	Data Pointer (Low Byte) for Addressing External Memory	82H

1.5 Flags and Program Status Word (PSW)

Flags are 1 bit storage registers that store and indicate the nature of the result that is generated by execution of certain instructions.

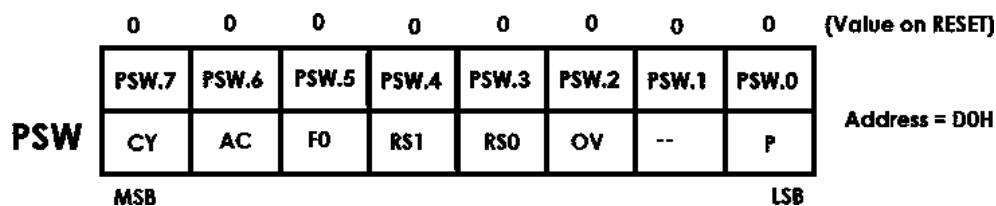


Figure 1.11: PSW register representation of 8051.

1.6 Stack Pointer

The stack is a section of RAM used by the CPU to store information temporarily. This information could be data or an address. The CPU needs this storage area since there are only a limited number of registers.

1.7 Data Pointer

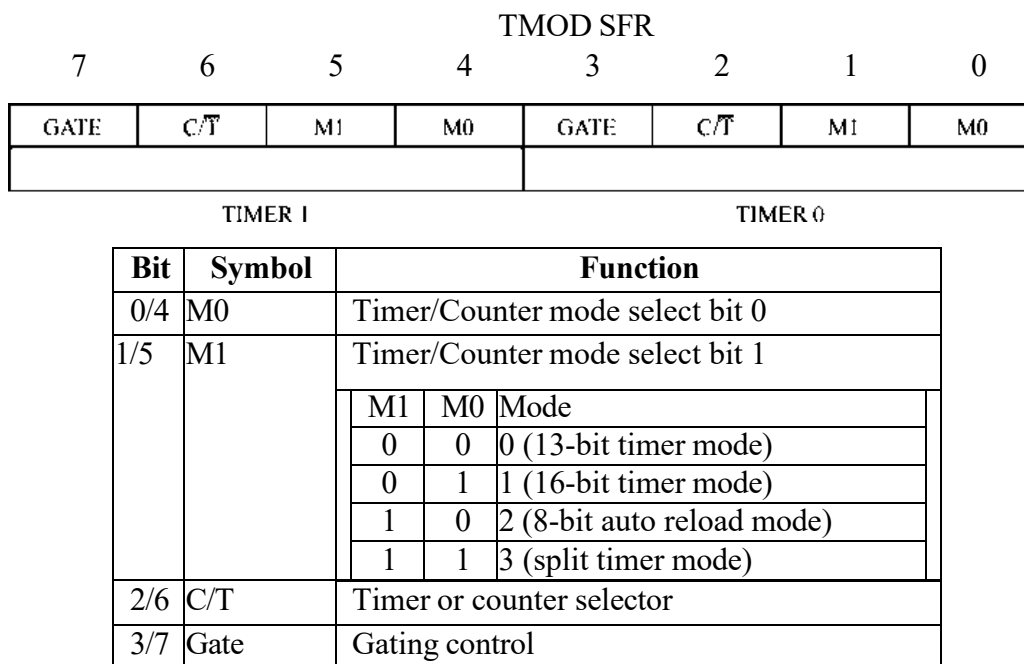
The Data Pointer, DPTR, is a special 16 bit register used to address the external code or external data memory.

1.8 Counters and Timers

The 8051 microcontroller has two 16 bit timers/counters called T0 and T1, which are provided for the general use of the program designer. These timers/counters may be programmed to count the internal pulses acting as a timer or programmed to count the external pulses as a counter.

Timers/counters are divided into two 8 bit registers called timer low (TL0, TL1) and high (TH0, TH1) bytes. The timers/counters action is controlled by bits status in the Timer Mode Control Register (TMOD), Timers/counters Control Register (TCON) and certain program instructions.

TMOD is dedicated for two timers/counters and has two duplicate 4 bit registers, each of which controls the action of one of the timers/counters. TCON has control bits & flags for the timers in upper nibble and control bits & flags for the external interrupts in the lower nibble. Figure 1.17 shows the bit assignments of the TMOD and TCON.



If the timer/counter is programmed as timer then it will count the internal clock frequency of the 8051 oscillator divided by 12d. For example if the crystal have frequency 12 MHz then the timer clock frequency is 1 micro hertz. The timer has four modes of the operations viz. Timer Mode 0, Timer Mode 1, Timer Mode 2 and Timer Mode 3. These modes are selected by the TMOD SFR bits, M0 and M1, shown in figure 1.17.

As stated earlier the difference between counting and timing is the source of the clock pulses to the timer/counter. So when timer/counter is used as a counter then external pulse from the pin T0 (P3.4) and pin T1 (P3.5) are supplied to controller.

1.9 Serial Data I/O

The serial port of 8051 is full duplex, i.e. it can transmit and receive simultaneously. The register SBUF to hold the data, register SCON to control the data communication,

SCON SFR							
7	6	5	4	3	2	1	0
SM0	SM1	SM2	REN	TB8	RB8	TI	RI

Bit	Symbol	Function		
7	SM0	Serial port mode 0		
6	SM1	Serial port mode 1		
		SM0	SM1	Mode
		0	0	0 (baud rate $f/2$)
		0	1	1 (baud rate variable)
		1	0	2 (baud rate $f/32$ or $f/64$)
		1	1	3 (baud rate variable)
5	SM2	Multiprocessor communication bit		
4	REN	Receive enable bit.		
3	TB8	Transmitted bit 8		
2	RB8	Received bit 8.		
1	TI	Transmit interrupt flag		
0	RI	Receive interrupt flag		

Figure 1.18: The bit assignments of the SCON.

Register PCON controls processor power down, sleep modes and serial data band rate. Only one bit of PCON is used with respect to serial communication. The seventh bit

1.10 Interrupts

An interrupt causes a temporary diversion of program execution in a similar sense to a program subroutine call, but an interrupt is triggered by some event, external to the currently operating program.

There are five interrupt sources for the 8051. Since the main RESET input can also be considered as an interrupt, six interrupts can be listed as below.

Interrupt	Flag	Vector address

System RESET	RST	0000H
External interrupt 0	IE0	0003H
Timer/counter 0	TF0	000BH
External interrupt 1	IE1	0013H
Timer/counter 1	TF1	001BH
Serial port	RI or TI	0023H

I/O ports structure :

The **8051 microcontroller** has **4 parallel I/O ports**, each **8 bits wide**, so a total of **32 I/O pins**. These ports are named **Port 0, Port 1, Port 2, and Port 3**. Each port has a specific **structure and function**.

General Port Structure

- Each port consists of **8 pins (P0.0–P0.7, P1.0–P1.7, etc.)**
- Each pin has:
 - A **latch** (to store output data)
 - A **driver/transistor**
 - An **input buffer**
- Ports can work as **input or output** (quasi-bidirectional)

Port-wise Structure and Functions

◆ Port 0 (P0)

- **Pins:** P0.0 – P0.7
- **Structure:**
 - **Open-drain** output (no internal pull-up resistors)
- **Functions:**
 - Acts as **multiplexed low-order address/data bus (AD0–AD7)** during external memory access
 - Can be used as I/O **only with external pull-up resistors**
- **Special Feature:** Dual-purpose port

◆ Port 1 (P1)

- **Pins:** P1.0 – P1.7
- **Structure:**
 - Has **internal pull-up resistors**
- **Functions:**
 - Used **only as general-purpose I/O**
- **Special Feature:** No alternate functions (in standard 8051)

◆ Port 2 (P2)

- **Pins:** P2.0 – P2.7
- **Structure:**
 - Has **internal pull-up resistors**
- **Functions:**
 - Acts as **high-order address bus (A8–A15)** for external memory
 - Can also be used as general-purpose I/O

◆ Port 3 (P3)

- **Pins:** P3.0 – P3.7
- **Structure:**
 - Has **internal pull-up resistors**
- **Functions:** Each pin has an **alternate function**

Pin Alternate Function

P3.0 RXD (Serial input)

P3.1 TXD (Serial output)

P3.2 INT0 (External interrupt 0)

P3.3 INT1 (External interrupt 1)

P3.4 T0 (Timer 0 input)

P3.5 T1 (Timer 1 input)

P3.6 WR (External data memory write)

P3.7 RD (External data memory read)

Summary Table

Port Pull-up Special Function

P0  No AD0–AD7

P1  Yes General I/O

P2  Yes A8–A15

P3  Yes Multiple alternate functions

Clock and Reset Circuit :

8051 Most often a quartz crystal oscillator is connected to inputs XTAL1 (Pin19) & XTAL2 (Pin18). The quartz crystal also needs two capacitors of 30pf. One side of each capacitor is connected to ground shown in figure.

The 8051 works the clock of 12MHz.

The microcontroller 8051 requires on power resets circuits for resetting the device. Pin 9 is reset pin. It is active high input. Upon applying a high pulse to this pin the micro controller will reset and terminate all activities. This is known as ON power

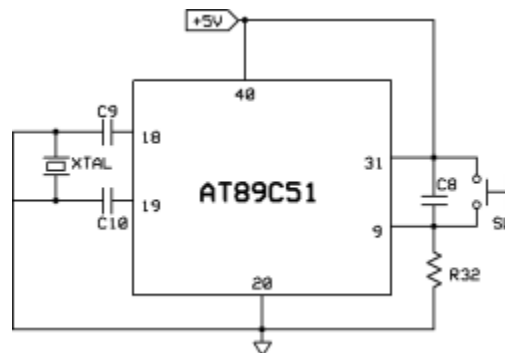


Figure 1.21: The clock and reset circuit

Timer/Counter :

In the **8051 microcontroller**, a **timer/counter** is a built-in hardware module used to **measure time delays** (timer mode) or **count external events** (counter mode).

1. Timers in 8051

The standard 8051 has **two timers/counters**:

- **Timer 0**
- **Timer 1**

Each can work as:

- **Timer** → counts internal clock pulses
- **Counter** → counts external pulses

2. Timer vs Counter

Mode	Counts	Input source
Timer	Machine cycles	Internal oscillator
Counter	External events	Pins T0 (P3.4), T1 (P3.5)

3. Timer Registers

(a) TMOD – Timer Mode Register

Used to select:

- Timer or counter
- Operating mode

TMOD = | GATE | C/T | M1 | M0 | GATE | C/T | M1 | M0 |
 Timer1 bits Timer0 bits

- **C/T = 0** → Timer mode
- **C/T = 1** → Counter mode

(b) TCON – Timer Control Register

P

Controls starting and stopping timers.

Important bits:

- **TR0 / TR1** → Start Timer 0 / Timer 1
- **TF0 / TF1** → Overflow flag

4. Timer Modes in 8051

Mode	Description
Mode 0	13-bit timer
Mode 1	16-bit timer
Mode 2	8-bit auto-reload
Mode 3	Split Timer 0 into two 8-bit timers

5. Timer Operation (Example: Mode 1 – 16 bit)

- Uses **THx** (high byte) and **TLx** (low byte)
- Counts from loaded value → **FFFFH**
- Overflow sets **TFx = 1**

6. Example: Timer 0 as Timer (Delay)

```
MOV TMOD, #01H    ; Timer 0, Mode 1
MOV TH0, #0FCH     ; Load initial value
MOV TL0, #66H
SETB TR0           ; Start Timer 0
```

```
HERE: JNB TF0, HERE ; Wait for overflow
CLR TR0
CLR TF0
```

7. Example: Timer 0 as Counter

```
MOV TMOD, #05H     ; Timer 0, Mode 1, Counter mode
SETB TR0           ; Start counting pulses on T0 (P3.4)
```

8. Applications

- Time delays
- Event counting
- Baud rate generation (Timer 1)
- Frequency measurement

Serial port :

The **8051 serial port** is a built-in **full-duplex UART** (Universal Asynchronous Receiver/Transmitter) used for **serial communication** with devices like PCs, sensors, or other microcontrollers.

1. Serial Port Pins

Pin	Function
P3.0 (RXD)	Receive data
P3.1 (TXD)	Transmit data

2. Serial Communication Registers

(a) SBUF – Serial Buffer Register

- Used for both **transmit and receive**
- Writing to SBUF → sends data
- Reading from SBUF → receives data

(b) SCON – Serial Control Register

SCON = | SM0 | SM1 | SM2 | REN | TB8 | RB8 | TI | RI |

Bit _p	Function
------------------	----------

Bit	Function
SM0, SM1	Select serial mode
SM2	Multiprocessor mode
REN	Enable reception
TB8	9th transmit bit
RB8	9th receive bit
TI	Transmit interrupt flag
RI	Receive interrupt flag

3. Serial Port Modes

Mode	Description	Baud Rate
Mode 0	Shift register (8-bit)	Fixed
Mode 1	8-bit UART	Variable
Mode 2	9-bit UART	Fixed
Mode 3	9-bit UART	Variable

👉 **Mode 1** is most commonly used.

4. Baud Rate Generation

- Generated using **Timer 1** (usually Mode 2)
- Baud rate depends on crystal frequency

Baud Rate Formula (Mode 1 & 3)

$$\text{Baud Rate} = \frac{2^{SMOD}}{32} \times \frac{f_{osc}}{12 \times (256 - TH1)}$$

5. Example: Serial Communication (Mode 1, 9600 baud)

Assumptions:

- Crystal = **11.0592 MHz**

MOV TMOD, #20H ; Timer 1, Mode 2

MOV TH1, #0FDH ; 9600 baud

MOV SCON, #50H ; Mode 1, REN enabled

SETB TR1 ; Start Timer 1

MOV SBUF, #'A' ; Send character A

HERE: JNB TI, HERE

CLR TI

6. Receiving Data Example

HERE: JNB RI, HERE ; Wait for data

MOV A, SBUF ; Read received data

CLR RI

7. Interrupts in Serial Port

- Uses **single interrupt vector**
- Triggered by **TI or RI**
- Interrupt vector address: **0023H**

8. Applications

- PC communication
- Debugging
- Wireless modules (Bluetooth, GSM)
- Data logging

Instruction set of Microcontroller 8051

Data Transfer Instructions

MOV

Syntax: MOV destination, source

MOV copies the value of source into destination. The value of source is not affected.

MOVX

Function: Move Data To/From External Memory (XRAM)

Syntax: MOVX *operand1*, *operand2*

MOVX moves a byte to or from External Memory into or from the Accumulator.

SWAP

Function: Swap Accumulator Nibbles

Syntax: SWAP A

Description: SWAP swaps bits 0-3 of the Accumulator with bits 4-7 of the Accumulator. This instruction is identical to executing "RR A" or "RL A" four times.

Operation: XCH **Function:** Exchange

Bytes Syntax: XCH A, *register*

Exchanges the value of the Accumulator with the value contained in register. Ex: XCH A, R1

PUSH

Function: Push Value Onto Stack

Syntax: PUSH

PUSH "pushes" the value of the specified *iram addr* onto the stack. PUSH first increments the value of the Stack Pointer by 1, then takes the value stored in *iram addr* and stores it in Internal RAM at the location pointed to by the incremented Stack Pointer.

POP

Function: Pop Value From Stack

Syntax: POP

Description: POP "pops" the last value placed on the stack into the *iram addr* specified. In other words, POP will load *iram addr* with the value of the Internal RAM address pointed to by the current Stack Pointer. The stack pointer is then decremented by 1.

Arithmetic Instructions

Operation: ADD, ADDC

Function: Add Accumulator, Add Accumulator With Carry

Description: Description: ADD and ADDC both add the value *operand* to the value of the Accumulator, leaving the resulting value in the Accumulator. The value *operand* is not affected. ADD and ADDC function identically except that ADDC adds the value of *operand* as well as the value of the Carry flag whereas ADD does not add the Carry flag to the result.

Operation: SUBB

Function: Subtract from Accumulator With Borrow

SUBB subtract the value of *operand* from the value of the Accumulator, leaving the resulting value in the Accumulator. The value *operand* is not affected.

The **Carry Bit (C)** is set if a borrow was required for bit 7, otherwise it is cleared. In other words, if the unsigned value being subtracted is greater than the Accumulator the Carry Flag is set.

Operation: MUL

Function: Multiply Accumulator by B

Syntax: MUL AB

Multiplies the unsigned value of the Accumulator by the unsigned value of the "B" register. The least significant byte of the result is placed in the Accumulator and the most-significant-byte is placed in the "B" register.

The **Carry Flag (C)** is always cleared.

Operation: DIV

Function: Divide Accumulator by B

Syntax: DIV AB

Divides the unsigned value of the Accumulator by the unsigned value of the "B" register. The resulting quotient is placed in the Accumulator and the remainder is placed in the "B" register. The **Carry flag (C)** is always cleared.

Operation: INC

Function: Increment Register

Syntax: INC *register*

INC increments the value of *register* by 1. If the initial value of *register* is 255 (0xFF Hex), incrementing the value will cause it to reset to 0. Note: The Carry Flag is NOT set when the value "rolls over" from 255 to 0.

In the case of "INC DPTR", the value two-byte unsigned integer value of DPTR is incremented. If the initial value of DPTR is 65535 (0xFFFF Hex), incrementing the value will cause it to reset to 0. Again, the Carry Flag is NOT set when the value of DPTR "rolls over" from 65535 to 0

Operation: DEC

Function: Decrement Register

Syntax: DEC *register*

DEC decrements the value of *register* by 1. If the initial value of *register* is 0, decrementing the value will cause it to reset to 255 (0xFF Hex). Note: The Carry Flag is NOT set when the value "rolls over" from 0 to 255.

Logical Instructions

Operation: ORL Function: Bitwise

OR

Syntax: ORL *operand1,operand2*

ORL does a bitwise "OR" operation between *operand1* and *operand2*, leaving the resulting value in *operand1*. The value of *operand2* is not affected. A logical "OR" compares the bits of each operand and sets the corresponding bit in the resulting byte if the bit was set in either of the

original operands, otherwise the resulting bit is cleared.

Operation: ANL Function: Bitwise

AND

Syntax: ANL *operand1*, *operand2*

ANL does a bitwise "AND" operation between *operand1* and *operand2*, leaving the resulting value in *operand1*. A logical "AND" compares the bits of each operand and sets the corresponding bit in the resulting byte only if the bit was set in both of the original operands, otherwise the resulting bit is cleared.

Operation: XRL

Function: Bitwise Exclusive OR

Syntax: XRL *operand1*, *operand2*

XRL does a bitwise "EXCLUSIVE OR" operation between *operand1* and *operand2*, leaving the resulting value in *operand1*. The value of *operand2* is not affected. A logical "EXCLUSIVE OR" compares the bits of each operand and sets the corresponding bit in the resulting byte if the bit was set in either (but not both) of the original operands, otherwise the bit is cleared.

Operation: CPL

Function: Complement Register

Syntax: CPL *operand*

CPL complements *operand*, leaving the result in *operand*. If *operand* is a single bit then the state of the bit will be reversed. If *operand* is the Accumulator

Operation: CLR

Function: Clear Register

Syntax: CLR *register*

CLR clears (sets to 0) all the bit(s) of the indicated register. If the register is a bit (including the carry bit), only the specified bit is affected. Clearing the Accumulator sets the Accumulator's value to 0.

Operation: RL

Function: Rotate Accumulator Left

Syntax: RL A

Shifts the bits of the Accumulator to the left. The left-most bit (bit 7) of the Accumulator is loaded into bit 0.

RR

Function: Rotate Accumulator Right

Syntax: RR A

Shifts the bits of the Accumulator to the right. The right-most bit (bit 0) of the Accumulator is loaded into bit 7.

Operation: RLC

Function: Rotate Accumulator Left Through Carry

Syntax: RLC A

Shifts the bits of the Accumulator to the left. The left-most bit (bit 7) of the Accumulator is loaded into the Carry Flag, and the original Carry Flag is loaded into bit 0 of the Accumulator. This function can be used to quickly multiply a byte by 2.

Operation: RRC

Function: Rotate Accumulator Right Through Carry

Syntax: RRC A

Shifts the bits of the Accumulator to the right. The right-most bit (bit 0) of the Accumulator is loaded into the Carry Flag, and the original Carry Flag is loaded into bit 7. This function can be used to quickly divide a byte by 2.

The **8051 microcontroller** has a rich instruction set that allows it to perform a variety of operations, including arithmetic, logical, data transfer, branching, and bit manipulation. Below is an overview of the key instructions in the 8051 instruction set.

1. Data Transfer Instructions:

These instructions are used to move data between registers, memory, and I/O ports.

Instruction	Description
MOV A, B	Move data from register B to register A.
MOV A, #data	Load immediate data into the accumulator A.
MOV Rn, #data	Load immediate data into register Rn (where n = 0 to 7).
MOV @R0, A	Move contents of accumulator A into the memory location pointed by register R0.
MOV A, @R0	Move the data pointed to by register R0 into the accumulator A.
MOV DPTR, #data	Load 16-bit immediate data into Data Pointer (DPTR).
MOVX A, @DPTR	Move data from external memory (pointed by DPTR) into accumulator A.
MOVX @DPTR, A	Move data from accumulator A into external memory (pointed by DPTR).

2. Arithmetic Instructions:

These instructions perform arithmetic operations like addition, subtraction, etc.

Instruction	Description
ADD A, B	Add the contents of register B to A.
ADD A, #data	Add immediate data to the accumulator A.
ADDC A, B	Add the contents of B with carry to A.
SUBB A, B	Subtract B from A with borrow.
INC A	Increment the accumulator A by 1.
INC Rn	Increment the register Rn by 1 (where n = 0 to 7).
DEC A	Decrement the accumulator A by 1.
DEC Rn	Decrement register Rn by 1 (where n = 0 to 7).
MUL AB	Multiply A by B, result in registers A and B.
DIV AB	Divide A by B, result in registers A (quotient) and B (remainder).

3. Logical Instructions:

These instructions perform logical operations like AND, OR, XOR, and NOT.

Instruction	Description
ANL A, B	Perform bitwise AND of A and B, and store the result in A.
ANL A, #data	Perform bitwise AND of A and immediate data, store in A.
ORL A, B	Perform bitwise OR of A and B, and store the result in A.
ORL A, #data	Perform bitwise OR of A and immediate data, store in A.
XRL A, B	Perform bitwise XOR of A and B, and store the result in A.
XRL A, #data	Perform bitwise XOR of A and immediate data, store in A.
CPL A	Complement (invert) the accumulator A.
CLR A	Clear the accumulator A (set it to 0).
CLR C	Clear the carry flag (C = 0).
SETB C	Set the carry flag (C = 1).

4. Branching Instructions:

These instructions control the flow of the program.

Instruction	Description
SJMP label	Short jump to a label (within -128 to +127 bytes).
LJMP label	Long jump to a label (full 16-bit address).
JMP @A+DPTR	Jump to the address contained in A and DPTR.
JC label	Jump if carry flag is set (C = 1).
JNC label	Jump if carry flag is not set (C = 0).

Instruction	Description
JZ label	Jump if zero flag is set ($Z = 1$).
JNZ label	Jump if zero flag is not set ($Z = 0$).
CALL label	Call a subroutine at the specified label.
RET	Return from a subroutine.
RETI	Return from an interrupt service routine.

5. Bit Manipulation Instructions:

These instructions allow manipulation of individual bits in the accumulator or registers.

Instruction Description

SETB bit	Set the specified bit to 1.
CLR bit	Clear the specified bit (set it to 0).
CPL bit	Complement the specified bit.
MOV bit, A	Move the accumulator (A) to the specified bit.
MOV A, bit	Move the specified bit to the accumulator (A).
ANL bit, A	AND the specified bit with the accumulator (A).
ORL bit, A	OR the specified bit with the accumulator (A).
XRL bit, A	XOR the specified bit with the accumulator (A).

6. Stack Instructions:

These instructions are used to manage the stack.

Instruction Description

PUSH direct	Push a direct memory location onto the stack.
POP direct	Pop the top value from the stack into a direct memory location.

7. Special Instructions:

These instructions perform special operations like setting the interrupt enable, managing the auxiliary carry, etc.

Instruction Description

NOP	No operation (does nothing but consumes one clock cycle).
SWAP A	Swap the upper and lower nibbles of the accumulator.
DA A	Decimal adjust the accumulator (used for BCD operations).
MOV PSW, A	Move the contents of A to the Program Status Word (PSW).
MOV A, PSW	Move the Program Status Word (PSW) to the accumulator.

8. Interrupt Instructions:

These instructions manage the interrupt system.

Instruction Description

EI	Enable interrupts.
DI	Disable interrupts.

Embedded C is a C programming language extension used to program microcontrollers and embedded systems (like 8051, AVR, PIC, ARM). It allows direct control of hardware such as ports, timers, interrupts, and memory.

Addressing modes :

The **8051 microcontroller** supports several **addressing modes**, which define **how operands are specified** in an instruction. These are very important for exams and programming.

Addressing Modes of 8051

◆ 1. Immediate Addressing

- The operand is **given directly in the instruction**.
- Indicated by # symbol.

Example:

P

MOV A, #25H
MOV R1, #10

◆ **2. Register Addressing**

- Operand is stored in a **register (R0–R7)**.
- Faster execution.

Example:

MOV A, R3
ADD A, R5

◆ **3. Direct Addressing**

- The **direct address of internal RAM or SFR** is specified.

Example:

MOV A, 30H ; Internal RAM
MOV P1, A ; SFR (Port 1)

◆ **4. Register Indirect Addressing**

- The address of the operand is stored in a **register (R0 or R1 only)**.
- Uses **@R0** or **@R1**.

Example:

MOV A, @R0
MOV @R1, A

◆ **5. Indexed Addressing**

- Used for **accessing program (code) memory**.
- Effective address = **A + DPTR** or **A + PC**.

Example:

MOVC A, @A+DPTR
MOVC A, @A+PC

◆ **6. Bit Addressing**

- Used to access **individual bits** in:
 - Bit-addressable RAM (20H–2FH)
 - Bit-addressable SFRs

Example:

SETB P1.0
CLR PSW.7

◆ **7. Implied (Implicit) Addressing**

- Operand is **implied**, not explicitly written.
- Commonly uses **Accumulator (A)**.

Example:

CLR A
CPL C

Summary Table

Addressing Mode	Example
Immediate	MOV A, #55H
Register	MOV A, R2
Direct	MOV A, 40H
Register Indirect	MOV A, @R0
Indexed	MOVC A, @A+DPTR
Bit	SETB P3.2
Implied	CLR A

Embedded C :

- ◆ What is Embedded C?
 - A subset/superset of C with additional features for hardware control
 - Used to write firmware for embedded devices
 - Programs interact directly with registers and peripherals
- ◆ Features of Embedded C
 - Fast execution
 - Low memory usage
 - Supports bit manipulation
 - Close to hardware
 - Portable across microcontrollers (with changes in headers)

- ◆ Differences: C vs Embedded C

C Language	Embedded C
Runs on PCs	Runs on microcontrollers
No hardware access	Direct hardware access
Uses OS	No OS (mostly bare-metal)
Unlimited memory (theory)	Limited RAM/ROM

The Embedded C program structure defines how a program is organized to run on a microcontroller (like 8051). It is slightly different from normal C because it includes hardware-specific definitions.

Basic Structure of an Embedded C Program

- ◆ 1. Header File Inclusion

Includes microcontroller-specific register definitions.
`#include <reg51.h> // For 8051 microcontroller`

- ◆ 2. Macro Definitions (Optional)

Used to define constants or pin connections.
`#define LED P1`

- ◆ 3. Global Variable Declaration

Variables accessible throughout the program.
`unsigned int count;`

- ◆ 4. Special Function Declarations

Used for bit-level and register-level access.
`sbit LED1 = P1^0;`

- ◆ 5. Function Prototypes

Declare functions before `main()`.
`void delay(void);`

- ◆ 6. Main Function

Execution starts here. Usually contains an infinite loop.

```
void main(void)
{
    while(1)
    {
        P
```

```

    LED1 = 1;
    delay();
    LED1 = 0;
    delay();
}
}

```

◆ 7. User-defined Functions

Functions like delay, LCD control, UART, etc.

```
void delay(void)
```

```

{
    int i, j;
    for(i=0;i<1000;i++)
        for(j=0;j<100;j++);
}

```

General Embedded C Program Structure (Flow)

Include header files

↓

Define macros

↓

Declare global variables

↓

Declare special registers/bits

↓

Function prototypes

↓

main() function

↓

Infinite loop

↓

User-defined functions

Example: Complete Embedded C Program (8051)

```
#include <reg51.h>
```

```
sbit LED = P1^0;
```

```
void delay(void);
```

```
void main()
```

```

{
    while(1)
    {
        LED = 1;
        delay();
        LED = 0;
        delay();
    }
}

```

P

```
void delay()
{
    unsigned int i;
    for(i=0;i<50000;i++);
}
```

Key Points (Exam-Oriented)

- main() is mandatory
- Program usually runs forever
- Hardware accessed using SFRs and bits
- No standard input/output (printf rarely used)

◆ Basic Structure of an Embedded C Program (8051 Example)

```
#include <reg51.h> // 8051 register definitions
```

```
void main(void)
{
    P1 = 0x00;    // Set Port 1 as output
    while(1)
    {
        P1 = 0xFF; // Turn ON LEDs
    }
}
```

◆ Embedded C Data Types

- char (8-bit)
- int (16-bit in 8051)
- long (32-bit)
- bit (single-bit data type, very important)
- sbit (used for SFR bit access)

Example:

```
sbit LED = P1^0; // LED connected to P1.0
bit flag;
```

◆ Special Keywords in Embedded C (8051)

- sbit – Access single bit
- sfr – Access special function register
- code, data, xdata, idata – Memory types
- interrupt – Interrupt function

◆ Example: LED Blinking Program (8051)

```
#include <reg51.h>
```

```
void delay(void);
```

```
void main()
{
    while(1)
    {
        P1 = 0xFF;
        delay();
        P1 = 0x00;
        delay();
    }
}
```

P

```

void delay()
{
    int i, j;
    for(i=0;i<1000;i++)
        for(j=0;j<100;j++);
}

```

- ◆ Applications of Embedded C
 - Washing machines
 - Microwave ovens
 - Robotics
 - Medical devices
 - Automobiles
 - IoT systems
- ◆ Advantages of Embedded C
 - Easy to learn
 - Hardware-level control
 - Faster development than Assembly
 - Reusable code

Program to Blink an LED

This ALP program toggles an LED connected to **P1.0** at regular intervals using a delay loop.

```

ORG 0H
MOV P1, #00H      ; Set all pins of Port 1 to low (turn off LED initially)
BLINK_LED:
    MOV P1, #01H   ; Set P1.0 to high (turn on LED)
    ACALL DELAY    ; Call delay subroutine
    MOV P1, #00H   ; Set P1.0 to low (turn off LED)
    ACALL DELAY    ; Call delay subroutine
    SJMP BLINK_LED ; Repeat the loop
DELAY:
    MOV R0, #255   ; Set delay count
DELAY_LOOP:
    DJNZ R0, DELAY_LOOP ; Decrement and loop
    RET           ; Return from delay
END

```